Autoregressive decomposition with parametrised distributions

```python
from torch.distributions import MixtureSameFamily, Categorical, Normal

BATCHSIZE = 128
N_DIM = 2
K = 10
iterations = 2000

# unconditional parameters for step 1; MLP for step 2
params_1 = nn.Parameter(0.1 * torch.randn(3 * K))
mlp_2 = nn.Sequential(
    nn.Linear(1, 64), nn.SiLU(),
    nn.Linear(64, 64), nn.SiLU(),
    nn.Linear(64, 3 * K),
)

def gmm(params):
    """K-component 1D GMM from a (..., 3K) parameter tensor."""
    logits, means, log_stds = params.chunk(3, dim=-1)
    return MixtureSameFamily(Categorical(logits=logits), Normal(means, log_stds.exp()))

optimizer = torch.optim.Adam([params_1, *mlp_2.parameters()], lr=1e-3)

for i in range(iterations):
    data, _ = make_moons(n_samples=BATCHSIZE, noise=0.05)
    x = torch.from_numpy(data).float()
    x_1, x_2 = x[:, 0], x[:, 1]

    # log p(x) = log p(x_1) + log p(x_2 | x_1)
    log_p_1 = gmm(params_1).log_prob(x_1)
    log_p_2 = gmm(mlp_2(x_1.unsqueeze(-1))).log_prob(x_2)
    loss = -(log_p_1 + log_p_2).mean()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

# Sample autoregressively
with torch.no_grad():
    x_1 = gmm(params_1).sample((10000,))
    x_2 = gmm(mlp_2(x_1.unsqueeze(-1))).sample()
    samples = torch.stack([x_1, x_2], dim=-1)
```

Variational Autoencoder (VAE)

```
BATCHSIZE = 128
N_DIM = 2
iterations = 2000
LATENT_DIM = 2
SIGMA_X = 0.1

encoder = nn.Sequential(
    nn.Linear(N_DIM, 64), nn.GELU(),
    nn.Linear(64, 64), nn.GELU(),
    nn.Linear(64, 2 * LATENT_DIM),
)
decoder = nn.Sequential(
    nn.Linear(LATENT_DIM, 64), nn.GELU(),
    nn.Linear(64, 64), nn.GELU(),
    nn.Linear(64, N_DIM),
)
params = list(encoder.parameters()) + list(decoder.parameters())
optimizer = torch.optim.Adam(params, lr=1e-3)

encoder.train(); decoder.train()
for i in range(iterations):
    data, _ = make_moons(n_samples=BATCHSIZE, noise=0.05)
    x = torch.from_numpy(data).float()

    mu_z, log_var_z = encoder(x).chunk(2, dim=-1)
    std_z = torch.exp(0.5 * log_var_z)
    z = mu_z + std_z * torch.randn_like(mu_z)
    mu_x = decoder(z)

    # -log p(x|z) for N(mu_x, SIGMA_X)   (constant dropped)
    recon = 0.5 * torch.sum((x - mu_x)**2, dim=-1) / SIGMA_X**2

    # KL(q(z|x) || N(0, I))
    kl = 0.5 * torch.sum(mu_z**2 + std_z**2 - log_var_z - 1, dim=-1)

    loss = (recon + kl).mean()
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

encoder.eval(); decoder.eval()
with torch.no_grad():
    z = torch.randn(10000, LATENT_DIM)
    samples = decoder(z)
```

Generative adversarial network (GAN)

```
BATCHSIZE = 128
N_DIM = 2
N_LATENT = 2
iterations = 2000

generator = nn.Sequential(
    nn.Linear(N_LATENT, 64), nn.SiLU(),
    nn.Linear(64, 64), nn.SiLU(),
    nn.Linear(64, N_DIM),
)
discriminator = nn.Sequential(
    nn.Linear(N_DIM, 64), nn.SiLU(),
    nn.Linear(64, 64), nn.SiLU(),
    nn.Linear(64, 1),
)

optimizer_G = torch.optim.Adam(generator.parameters(), lr=1e-3, betas=(0.9, 0.999))
optimizer_D = torch.optim.Adam(discriminator.parameters(), lr=1e-3, betas=(0.9, 0.999))
loss_fn = nn.BCEWithLogitsLoss()

generator.train(); discriminator.train()
for i in range(iterations):
    data, _ = make_moons(n_samples=BATCHSIZE, noise=0.05)
    x_real = torch.from_numpy(data).float()
    z = torch.randn(BATCHSIZE, N_LATENT)
    x_fake = generator(z)

    # discriminator step
    D_real = discriminator(x_real)
    D_fake = discriminator(x_fake.detach())  # block gradient flow
    loss_D = (loss_fn(D_real, torch.ones_like(D_real))
              + loss_fn(D_fake, torch.zeros_like(D_fake)))
    optimizer_D.zero_grad()
    loss_D.backward()
    optimizer_D.step()

    # generator step
    D_fake = discriminator(x_fake)
    loss_G = loss_fn(D_fake, torch.ones_like(D_fake))
    optimizer_G.zero_grad()
    loss_G.backward()
    optimizer_G.step()

generator.eval()
with torch.no_grad():
    z = torch.randn(10000, N_LATENT)
    samples = generator(z)
```

Normalising flow

```python
import zuko

BATCHSIZE = 128
N_DIM = 2
iterations = 1000

transform = zuko.flows.NSF(features=N_DIM, transforms=5, hidden_features=(32, 32)).transform
optimizer = torch.optim.Adam(transform.parameters(), lr=1e-3)

transform.train()
for i in range(iterations):
    data, _ = make_moons(n_samples=BATCHSIZE, noise=0.05)
    x = torch.from_numpy(data).float()

    z, log_det = transform().call_and_ladj(x)       # forward flow transform
    log_prob_base = -0.5 * torch.sum(z**2, dim=-1)  # base distribution likelihood; constant term dropped
    log_prob = log_prob_base + log_det              # target distribution likelihood
    loss = -log_prob.mean()

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

transform.eval()
with torch.no_grad():
    z = torch.randn(10000, N_DIM)  # sample from base distribution
    samples = transform().inv(z)   # inverse flow transform
```

Conditional Flow Matching (CFM)

```python
from torchdiffeq import odeint

BATCHSIZE = 1024
N_DIM = 2
iterations = 2000

N_FREQS = 8
freqs = 2 * torch.pi * torch.arange(1, N_FREQS + 1).float()
def time_embed(t): # t -> [sin(2πk t), cos(2πk t)] for k = 1..K
    return torch.cat([torch.sin(t * freqs), torch.cos(t * freqs)], dim=-1)

velocity_net = nn.Sequential(
    nn.Linear(N_DIM + 2 * N_FREQS, 64), nn.GELU(),
    nn.Linear(64, 64), nn.GELU(),
    nn.Linear(64, 64), nn.GELU(),
    nn.Linear(64, N_DIM),
)
optimizer = torch.optim.Adam(velocity_net.parameters(), lr=1e-3)

velocity_net.train()
for i in range(iterations):
    data, _ = make_moons(n_samples=BATCHSIZE, noise=0.05)
    x0  = torch.from_numpy(data).float()

    # random base distribution and time point
    eps = torch.randn_like(x0)
    t   = torch.rand(BATCHSIZE, 1)

    # conditional target velocity
    x_t = (1 - t) * x0 + t * eps
    v_t = eps - x0

    # unconditional predicted velocity
    v_pred = velocity_net(torch.cat([x_t, time_embed(t)], dim=-1))
    loss = ((v_pred - v_t) ** 2).mean()  # CFM loss

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

velocity_net.eval()
with torch.no_grad():
    def velocity_func(t, x):
        t_batch = t * torch.ones(x.shape[0], 1)
        return velocity_net(torch.cat([x, time_embed(t_batch)], dim=-1))

    eps = torch.randn(10000, N_DIM)
    samples = odeint(velocity_func, eps, torch.tensor([1., 0.]), method='dopri5', atol=1e-5, rtol=1e-5)[-1]
```

Link to coding exercises:

github.com/spinjo/ml-lecture-ippp-2026